

HW2: Stream Cyphers

CYB-402

Tyler Vander Mooren

1.1.

$$\Sigma = \{a, b, \dots, z, \ddot{a}, \mathfrak{b}, \dot{c}\}$$

Work in ring Z_{29}

Keystream: $s_1, s_2, \dots, s_i \in Z_{29}$ – Exactly analogous to Z_2

1.2. For plaintext $x_i \in Z_{29}$ and keystream $s_i \in Z_{29}$:

Encrypt: $y_i \equiv x_i + s_i \pmod{29}$

Decrypt: $x_i \equiv y_i - s_i \pmod{29}$

1.3. since the text is equal to the key – TOP w/same text

$$s_i = x_i, \text{ so } y_i = x_i + s_i \pmod{29}$$

$$y_i = x_i + x_i \pmod{29}$$

$$\text{We get: } y_i = 2x_i \pmod{29}$$

i	plain	x_i	$2x_i \pmod{29}$	Cipher
1	a	0	0	a
2	n	13	26	$\ddot{a}$
3	a	0	0	a
4	l	11	22	w
5	p	15	1	b
6	h	7	14	o
7	a	0	0	a
8	b	1	2	c
9	e	4	8	i
10	t	19	9	j
11	w	22	45	p
12	i	8	16	q
13	t	19	9	j
14	h	7	14	o
15	t	19	9	j
16	w	22	15	p
17	e	4	8	i
18	n	13	26	$\ddot{a}$
19	t	19	9	j

20	y	24	29	t
21	n	13	26	ä
22	i	8	16	q
23	n	13	27	ä
24	e	4	8	i
25	l	11	22	w
26	e	4	8	i
27	t	19	9	j
28	t	19	9	j
29	e	4	8	i
30	r	17	5	f
31	s	18	7	h
32	i	8	16	q
33	s	18	7	h
34	w	22	15	p
35	e	4	8	i
36	i	8	16	q
37	r	17	5	f
38	d	3	6	g

Plaintext: "an alphabet with twenty nine letters is weird" -> remove white spaces

""analphabettwithtwentyninelettersisweird"

Ciphertext: "ääawboacijpqjojpjätäqäiwijjifhghpiqfg"

Test decryption: ä -> y = 26, key s = 13.

$$x \equiv 26 - 13 \equiv 13 \pmod{29}$$

$$= 13 \rightarrow n$$

Matches original n

2. If the 256-bit key is reused periodically across large volumes of data, an attacker who knows any plaintext segment for one period can recover the keystream for that period and thus decrypt all other ciphertext segments that used the same keystream.

1. Take known plaintext P_1 and align it to a keystream repetition boundary (e.g., you know the first 8 bytes are a PNG header, or a HTTP request line, or a predictable protocol header).

2. Recover the keystream segment for that position

$$K = C_1 \oplus P_1$$

3. Since the 256-bit key is reused periodically, this recovered segment of the keystream will repeat, exposing the full keystream for any message. Decrypt any other ciphertexts that used the same keystream segment at that offset

For any ciphertext C_2 that used the same K at the same point, produce tampered ciphertext

$$C_2' = C_2 \oplus E$$

On Decryption:

$$P_2' = C_2' \oplus K = (C_2 \oplus E) \oplus K = P_2 \oplus E,$$

Which yields the original plaintext. With the full keystream, the attack can with any new ciphertext message find the plaintext by XORing it with the keystream.

3.1 A 1GB CD is enough to supply at most 1 GB of OTP key material, which is enough to encrypt 1 GB of plaintext exactly once. A 1 GB could be used but the generation, securely delivering, tracking, preventing reuse, and the destruction of the CD-ROM is only workable for very limited, high-assurance scenarios.

Key Generation

Requirement: OTP needs truly random bits so a TRNP is mandatory

Use a validated hardware TRNG and generate it in a controlled, auditable environment.

Generation throughput will matter, slower TRNGs can take hours to produce gigabytes.

3.2

Key Distribution

Requirement: Alice must securely get the random bits to bob.

This can be done by physically transporting the CD using a trusted courier or splitting the key and sending shares via independent routes.

However, if the CD at any point is copied or lost it compromises all ciphertexts that use that key. Synchronization must be exact to avoid accidental reuse.

3.3

Storage of the key during the life cycle

Goal: keep confidentiality, integrity, and any tampering evidence while also ensuring access to the authorized parties

With a CD-ROM it offers immutability but is copyable and fragile. Using physical vaults with strict access controls reduces risk but adds operational burden. Suggestions would be to have strict check-in/out, multi-person access controls, and one time read procedures. Also using precise indexing counters to guarantee every key bit is used only once. The threat remains the same as distribution and if insiders copied the CD-ROM, or other actors then it is compromised. The actual breaking or degradation of the CD-ROM can happen and bookkeeping errors leading to reuse can happen.

3.4.

Storage of the key after the life cycle

Goal: securely and verifiably destroy all copies after use. Its destruction is witnessed and destructions logs is the standard.

Must find and destroy all backup copies, a simple file deletion or wait till it degrades will not work.

4.1. From the lecture notes we get $s_{i+3} \equiv s_{i+1} \oplus s_i \text{ mod } 2$

$$\text{Full period} = 2^3 - 1 = 7$$

$$(p_0, p_1, p_2) = (1, 1, 0)$$

$$(s_0, s_1, s_2) = (1, 0, 1)$$

One full period

$s_3 = s_0 \oplus s_1 =$	$1 \oplus 0 =$	1
$s_4 = s_1 \oplus s_2 =$	$0 \oplus 1 =$	1
$s_5 = s_2 \oplus s_3 =$	$1 \oplus 1 =$	0
$s_6 = s_3 \oplus s_4 =$	$1 \oplus 1 =$	0
$s_7 = s_4 \oplus s_5 =$	$1 \oplus 0 =$	1
$s_8 = s_5 \oplus s_6 =$	$0 \oplus 0 =$	0
$s_9 = s_6 \oplus s_7 =$	$0 \oplus 1 =$	1

at $i = 7$ the 3-bit state cycles back to the state of the 7 length sequence:

[1,0,1,1,0,0 | 1,0,1,1,1,0,0 ...]

4.2.

$$\text{Full period} = 2^3 - 1 = 7$$

$$(p_0, p_1, p_2) = (1, 1, 0)$$

$$(s_0, s_1, s_2) = (0, 1, 0)$$

One full period

$s_3 = s_0 \oplus s_1 =$	$0 \oplus 1 =$	1
$s_4 = s_1 \oplus s_2 =$	$1 \oplus 0 =$	1
$s_5 = s_2 \oplus s_3 =$	$0 \oplus 1 =$	1
$s_6 = s_3 \oplus s_4 =$	$1 \oplus 1 =$	0
$s_7 = s_4 \oplus s_5 =$	$1 \oplus 1 =$	0
$s_8 = s_5 \oplus s_6 =$	$1 \oplus 0 =$	1
$s_9 = s_6 \oplus s_7 =$	$0 \oplus 0 =$	0

at $i = 7$ the 3-bit state cycles back to the state of the 7 length sequence:

[0,1,0,1,1,1,0 | 0,1,0,1,1,1,0 ...]

4.3. The two sequences are related since they are the same m-sequence phase shift.

(0,1,0,1,1,1,0) is a rotation of (1,0,1,1,0,0). If you rotate the first sequence by 6 positions you get the second, and the second is the first shifted by -1 position. Since the initial state lies on the single length of $2^3 - 1$, different seeds just pick different starting phases of the same length sequence.

5.

How the cipher can be attacked – Period based attack

$$c_i = p_i \oplus s_i, \quad p_i, c_i, s_i \in \{0,1\}$$

period $T \in [150, 200]$

What Oscar has to know:

Needs one full period of the keystream $s_0, \dots, s_{T-1}$

Needs known plaintext of length at least T aligned to a segment of ciphertext.
(targeting headers, predictable formats, etc).

How he can decrypt based on the latter:

Get the Keystream:

Given a known plaintext windows $[a, a + L - 1]$ with $L \geq T$:

Extract s where plaintext is known $s_i = c_i \oplus p_i$ for those $i = a, \dots, a + L - 1$

If the known plaintext is at least one full period, then Oscar has $s_0, \dots, s_{T-1}$.

Because the keystream repeats with period T ,

$S = (s_0, s_1, \dots, s_{T-1})$ with $s_j = s_{a+j}$ for $j=0, \dots, T-1$.

$$s_i = s_i \pmod{T}.$$

Now he can Decrypt every bit with: $p_i = c_i \oplus s_i \pmod{T}$ for all i .

6.

Degree 7 LFSR

Feedback polynomial: $x^7 + x + 1 \rightarrow$ new bit = $b_0 \oplus b_1$

State is 7 bits $[t_6, t_5, t_4, t_3, t_2, t_1, t_0]$ rightmost is b_0

Each Clock

Output $z_t = b_0$

Feedback $f = b_0 \oplus b_1 \pmod{2}$

Shift right and insert f on the left

$[b_6 \dots b_0] = [f, b_6, b_5, b_4, b_3, b_2, b_1]$

Initial state: $0x7F = 127$ decimal = $0111\ 1111$ binary

Drop left most 0 to get 7 least significant bits of $0x7F$:

1111111 is the initial state $\rightarrow 7F$

1111111 $\rightarrow$ out 1	$f = 1 \oplus 1 = 0$	0 111111
0111111 $\rightarrow$ out 1	$f = 1 \oplus 1 = 0$	0 011111
0011111 $\rightarrow$ out 1	$f = 1 \oplus 1 = 0$	0 001111
0001111 $\rightarrow$ out 1	$f = 1 \oplus 1 = 0$	0 000111
0000111 $\rightarrow$ out 1	$f = 1 \oplus 1 = 0$	0 000011
0000011 $\rightarrow$ out 1	$f = 1 \oplus 1 = 0$	0 000001
0000001 $\rightarrow$ out 1	$f = 1 \oplus 0 = 1$	1 000000
1000000 $\rightarrow$ out 0	$f = 0 \oplus 0 = 0$	0 100000
0100000 $\rightarrow$ out 0	$f = 0 \oplus 0 = 0$	0 010000
0010000 $\rightarrow$ out 0	$f = 0 \oplus 0 = 0$	0 001000
0001000 $\rightarrow$ out 0	$f = 0 \oplus 0 = 0$	0 000100
0000100 $\rightarrow$ out 0	$f = 0 \oplus 0 = 0$	0 000010
0000010 $\rightarrow$ out 0	$f = 0 \oplus 1 = 1$	1 000001
1000001 $\rightarrow$ out 1	$f = 1 \oplus 0 = 1$	1 100000
1100000 $\rightarrow$ out 0	$f = 0 \oplus 0 = 0$	0 110000
0110000 $\rightarrow$ out 0	$f = 0 \oplus 0 = 0$	0 011000

First 16 output bits = $1111110\ 00000100$

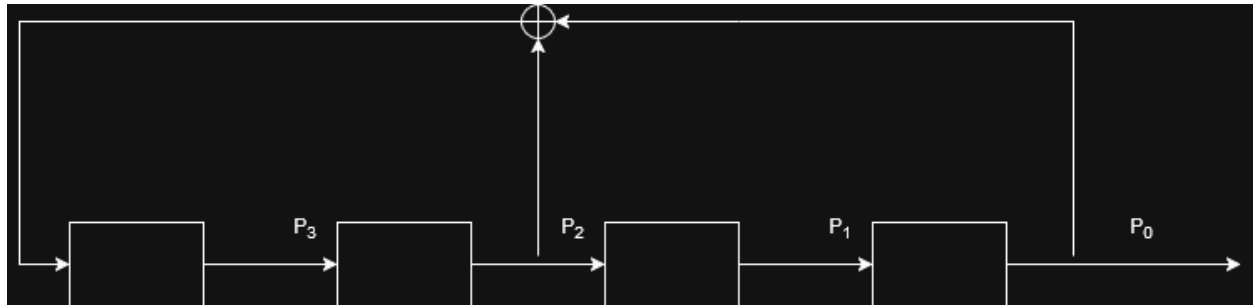
First two bytes =

$(1111110) \rightarrow 254$ decimal $\rightarrow 0xFE$

(00000100) -> 2 decimal -> 0x02

7.1. $x^4 + x^2 + 1$

$p_2 = p_1 = p_0 = 1$



$X^4 + X^2 + 1$ can be reducible -> $(x^2 + x + 1)^2$. The 15 nonzero states split into cycles of lengths 6, 6, and 3 depending on the seed.

Run [0 0 0 1] state = 1: states 1 -> 8 -> 4 -> 10 -> 5 -> 2 -> 1, outputs: 1 0 0 0 1 0 (period 6)

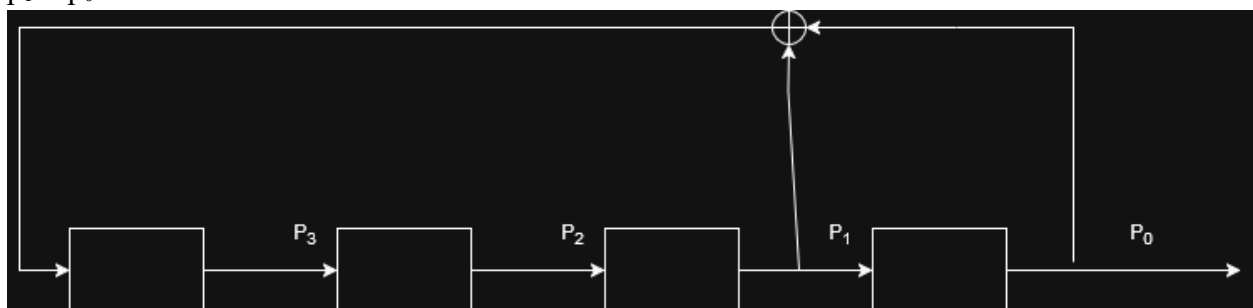
Run [0 0 1 1] state = 3: states 3 -> 9 -> 12 -> 14 -> 15 -> 7 -> 3, outputs: 1 1 0 0 1 1 (period 6)

Run [0 1 1 0] state = 6: states 6 -> 11 -> 13 -> 6, outputs 0 1 1 (period 3)

Polynomial is reducible with sequence length depending on initial state.

7.2. $x^4 + x + 1$

$p_1 = p_0 = 1$



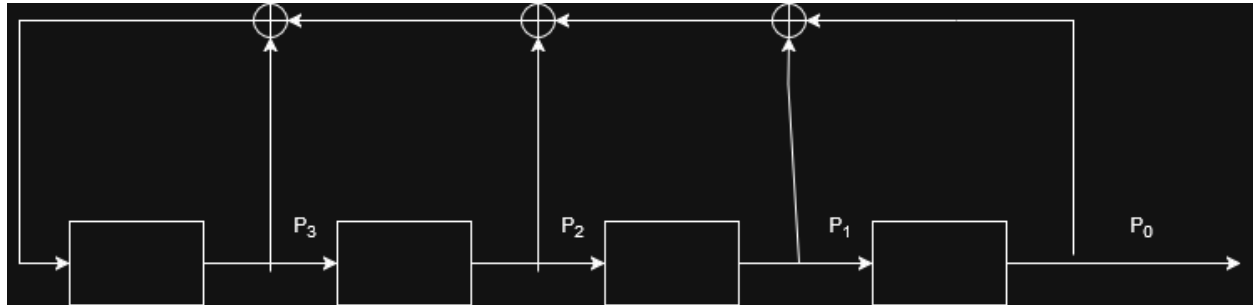
Period = $2^4 - 1 = 15$ cycles

Run [0 0 0 1] outputs: 1 0 0 0 1 0 0 1 1 0 1 0 1 1 1 (period 15)

Polynomial is primitive with maximum length of 15 for all nonzero initial states.

7.3. $x^4 + x^3 + x^2 + x + 1$

$$p_3 = p_2 = p_1 = p_0 = 1$$



Run [0 0 0 1]: outputs 1 0 0 0 1 (period 5)

Run [0 0 1 0]: outputs 0 1 0 0 1 (period 5)

Run [0 0 1 1]: outputs 1 1 0 0 0 (period 5)

Polynomial is irreducible but not primitive with sequence length of 5 for any nonzero initial states.

8.

$$P = 1001001001101101100100100110$$

$$C = 1011110000110001001010110001$$

Recover the keystream first $\rightarrow S = C \oplus P$

$$S = 0010111001011100101110010111$$

Get the LFSR degree m by testing small recurrences

Look for smallest m and taps $p_0, \dots, p_{m-1} \in \{0, 1\}$ s.i.

$$S_{i+m} \equiv \sum_{j=0}^{m-1} p_j S_{i+j} \pmod{2}, \text{ for all } i \text{ where it's defined.}$$

Try $m = 1$:

$$S_{i+1} = p_0 S_i$$

If $p_0 = 0$ then all later bits would be 0, not true

If $p_0 = 1$ then $S_{i+1} = S_i$, but $S_1 = 0$ and $S_2 = 1$ contradict that $\rightarrow m \neq 1$.

Try $m = 2$:

$$S_{i+2} = p_1 S_{i+1} \oplus p_0 S_i$$

$i = 0$:

$$s_2 = 1 = p_1 s_1 \oplus p_0 s_0 = p_1 * 0 \oplus p_0 * 0 = 0 \text{ contradiction } \rightarrow m \neq 2$$

Try $m = 3$:

$$s_{i+3} = p_2 s_{i+2} \oplus p_1 s_{i+1} \oplus p_0 s_i$$

$i = 0, 1, 2$ for p_2, p_1, p_0 :

$$i = 0: s_3 = 0 = p_2 s_2 \oplus p_1 s_1 \oplus p_0 s_0 = p_2 * 1 \oplus p_1 * 0 \oplus p_0 * 0 \rightarrow p_2 = 0$$

$$i = 1: s_4 = 1 = p_2 s_3 \oplus p_1 s_2 \oplus p_0 s_1 = 0 * 0 \oplus p_1 * 1 \oplus p_0 * 0 \rightarrow p_1 = 1$$

$$i = 2: s_5 = 1 = p_2 s_4 \oplus p_1 s_3 \oplus p_0 s_2 = 0 * 1 \oplus 1 * 0 \oplus p_0 * 1 \rightarrow p_0 = 1$$

so the recurrence is:

$$s_{i+3} = s_{i+1} \oplus s_i, p_2 = 0, p_1 = 1, p_0 = 1$$

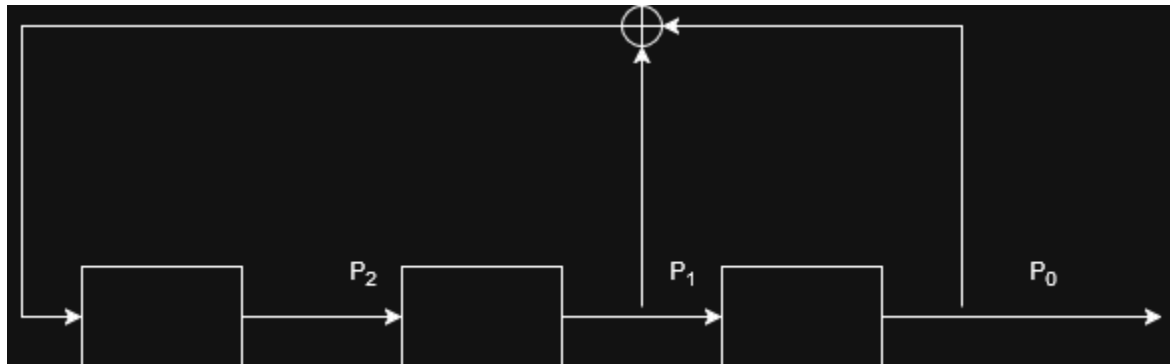
$$m = 3$$

8.1 degree $m = 3$

8.2 initial state = (0,0,1)

8.3 feedback coefficients: $p_1 = p_0 = 1, p_2 = 0$, Polynomial: $x^3 + x + 1$

8.4



Using $s_{i+3} = s_{i+1} \oplus s_i$ and seed (0, 0, 1) - Verify

$$i = 0, s_3 = s_1 \oplus s_0 = 0 \oplus 0 = 0. \text{ Correct}$$

$$i = 1, s_4 = s_2 \oplus s_1 = 1 \oplus 0 = 1. \text{ Correct}$$

$$i = 2, s_5 = s_3 \oplus s_2 = 0 \oplus 1 = 1. \text{ Correct}$$

$$i = 3, s_6 = s_4 \oplus s_3 = 1 \oplus 0 = 1. \text{ Correct}$$

$$i = 4, s_7 = s_5 \oplus s_4 = 1 \oplus 1 = 0. \text{ Correct}$$

Matches the recovered keystream